

Code Review Report

Language: python | Review mode: hybrid | Model: llama-3.3-70b-versatile | Generated: 2026-08-13 17:28 UTC

Overall Assessment

The code review has identified 12 findings with 9 critical and 3 high severity issues. These issues include hardcoded passwords and API keys, SQL injection vulnerabilities, and use of eval and exec functions. It is essential to address these findings to prevent security vulnerabilities and ensure the code's integrity.

Merge readiness: Not ready for merge due to critical and high severity findings

Code health: 0/100

Findings Summary

Total findings: 12

Risk score: 45

Severity breakdown: Low 0, Medium 0, High 3, Critical 9

Type	Severity	Line	Message	Recommendation
Hardcoded Password	High	1	password = "admin123"	Store passwords in environment variables.
Hardcoded API Key	High	3	api_key = "123456789"	Store secrets securely.
SQL Injection	Critical	7	query = "SELECT * FROM users WHERE name='" + name + "'"	Use parameterized queries.
Dangerous Dynamic Execution	Critical	9	eval(input())	Avoid dynamic code execution.
Dangerous Dynamic Execution	Critical	11	exec("print('Hello')")	Avoid dynamic code execution.
Command Injection	Critical	15	subprocess.run("dir", shell=True)	Do not use shell=True with user input.
Security	Critical	1	Hardcoded password	Use environment variables or a secure secrets management system to store sensitive data
Security	Critical	2	Hardcoded API key	Use environment variables or a secure secrets management system to store sensitive data
Security	Critical	5	SQL injection vulnerability	Use parameterized queries or prepared statements to prevent SQL injection attacks
Security	Critical	6	Use of eval function	Avoid using eval function and instead use safer alternatives such as ast.literal_eval or json.loads

Type	Severity	Line	Message	Recommendation
Security	Critical	7	Use of exec function	Avoid using exec function and instead use safer alternatives such as subprocess.run with a list of arguments
Security	High	10	Use of shell=True with subprocess.run	Avoid using shell=True and instead pass a list of arguments to subprocess.run to prevent shell injection attacks

Remediation Roadmap

Finding	Severity	Impact	Suggested Action
SQL Injection	Critical	High risk of data breach	Replace string concatenation with parameterized queries
Dangerous Dynamic Execution	Critical	High risk of code injection attacks	Replace eval and exec with safer alternatives
Command Injection	Critical	High risk of command injection attacks	Replace shell=True with a list of arguments in subprocess.run
Hardcoded Password	High	High risk of password exposure	Store passwords in environment variables
Hardcoded API Key	High	High risk of API key exposure	Store API keys in environment variables
Use of shell=True with subprocess.run	High	High risk of shell injection attacks	Replace shell=True with a list of arguments in subprocess.run

Source Code Snapshot

```
password = "admin123"

api_key = "123456789"

name = input()

query = "SELECT * FROM users WHERE name='" + name + "'"

eval(input())

exec("print('Hello')")

import subprocess

subprocess.run("dir", shell=True)
```